Grafana Loki

+ Grafana + Alloy

Installation native sur Debian 13 (sans Docker)

Centralisation des logs d'infrastructure

Projet IUTO — Documentation technique
Avril 2026

Table des matières

Table des matières 1
Architecture de la stack 2
Ports réseau 3
Installation de Loki 3
Création de l'utilisateur système 4
Téléchargement du binaire 4
Création des répertoires 4
Configuration /etc/loki/config.yaml 4
Service systemd /etc/systemd/system/loki.service 5
Installation de Grafana 6
Connexion Loki comme source de données 7
Installation d'Alloy (agent de collecte) 7
Installation via le dépôt APT Grafana 8
Configuration /etc/alloy/config.alloy 8
Permissions et démarrage 8
Interface de debug 9
Requêtes LogQL essentielles 9
Erreurs rencontrées et solutions 10
Erreur 1 : Erreur de format pour exec() 11
Erreur 2 : unknown short flag '-c' 11
Erreur 3 : status=203/EXEC 11
Erreur 4 : logcli au lieu de loki 11
Erreur 5 : unknown unit "h" in duration 11
Erreur 6 : delete-request-store should be configured 11
Erreur 7 : allow_structured_metadata supprimé 11
Erreur 8 : relabel_rules = [] capsule error (Alloy) 12
Récapitulatif des fichiers importants 12
Commandes de vérification 13

Architecture de la stack

La stack de centralisation des logs repose sur trois composants installés nativement (sans Docker) sur le serveur SRV-LOG (10.200.1.80, Debian 13) :

— Loki : moteur de stockage et d'indexation des logs (port 3100)

— Grafana : interface web de visualisation et de requêtage (port 3000)

— Alloy : agent de collecte installé sur chaque machine, successeur de Promtail

Le flux est le suivant : chaque serveur exécute un agent Alloy qui pousse ses logs vers Loki via HTTP (port 3100). Grafana se connecte à Loki pour permettre la consultation et la recherche dans les logs.

Ports réseau

Service	Port	Protocole	Usage
Loki HTTP API	3100	TCP	Réception des logs (push)
Loki gRPC	9096	TCP	Communication interne
Grafana	3000	TCP	Interface web
Alloy metrics	12345	TCP	Interface debug Alloy

Installation de Loki

Création de l'utilisateur système

```
useradd --system --no-create-home --shell /bin/false loki
```

Téléchargement du binaire

Télécharger le binaire depuis les releases GitHub. Vérifier l'architecture avec uname -m (doit retourner x86_64 pour amd64) :

```
# Vérifier l'architecture
uname -m

LOKI_VERSION="v3.4.3"
wget https://github.com/grafana/loki/releases/download/
  ${LOKI_VERSION}/loki-linux-amd64.zip
unzip loki-linux-amd64.zip
chmod +x loki-linux-amd64
mv loki-linux-amd64 /usr/local/bin/loki

loki --version
```

Attention : ne pas confondre loki-linux-amd64.zip (le serveur) avec logcli-linux-amd64.zip (le client CLI) ou promtail-linux-amd64.zip (l'ancien agent). Télécharger le bon fichier.

Création des répertoires

```
mkdir -p /etc/loki
mkdir -p /var/lib/loki/{chunks,index,cache,wal,compactor,rules,rules-temp}
chown -R loki:loki /var/lib/loki
```

Configuration /etc/loki/config.yaml

Configuration fonctionnelle pour Loki v3, compatible avec le schéma TSDB et la rétention de 30 jours :

```
auth_enabled: false

server:
  http_listen_port: 3100
  grpc_listen_port: 9096

common:
  instance_addr: 127.0.0.1
  path_prefix: /var/lib/loki
  storage:
    filesystem:
      chunks_directory: /var/lib/loki/chunks
      rules_directory: /var/lib/loki/rules
  replication_factor: 1
  ring:
    kvstore:
      store: inmemory

query_range:
  results_cache:
    cache:
      embedded_cache:
        enabled: true
        max_size_mb: 100

schema_config:
  configs:
    - from: 2024-01-01
      store: tsdb
      object_store: filesystem
      schema: v13
      index:
        prefix: index_
        period: 24h

limits_config:
  retention_period: 720h

compactor:
  working_directory: /var/lib/loki/compactor
  retention_enabled: true
  delete_request_store: filesystem
```

Service systemd /etc/systemd/system/loki.service

```
[Unit]
Description=Grafana Loki Log Aggregation System
Documentation=https://grafana.com/docs/loki/latest/
After=network.target

[Service]
Type=simple
User=loki
Group=loki
ExecStart=/usr/local/bin/loki \
  --config.file=/etc/loki/config.yaml
ExecReload=/bin/kill -HUP $MAINPID
Restart=on-failure
RestartSec=5
LimitNOFILE=65536
```

```
[Install]
WantedBy=multi-user.target
systemctl daemon-reload
systemctl enable --now loki
systemctl status loki

# Vérification
curl http://localhost:3100/ready
```

Important : utiliser --config.file= (double tiret).
Le simple tiret -config.file= ne fonctionne plus en Loki v3.

Installation de Grafana

```
apt install -y apt-transport-https software-properties-common
wget -q -O - https://apt.grafana.com/gpg.key \
  | gpg --dearmor -o /etc/apt/keyrings/grafana.gpg
echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] \
  https://apt.grafana.com stable main" \
  | tee /etc/apt/sources.list.d/grafana.list

apt-get update && apt-get install -y grafana

systemctl enable --now grafana-server
```

Accéder à Grafana : http://10.200.1.80:3000 (identifiants par défaut : admin / admin).

Connexion Loki comme source de données

1. Aller dans Connections > Data Sources > Add data source > Loki
2. URL : http://localhost:3100
3. Cliquer sur Save & test
4. Le message « Data source connected » confirme que Grafana peut lire les logs dans Loki

Installation d'Alloy (agent de collecte)

Grafana Alloy remplace Promtail (officiellement en fin de vie depuis mars 2026). Il est installé sur chaque machine dont on veut centraliser les logs.

Installation via le dépôt APT Grafana

```
apt install -y gpg
mkdir -p /etc/apt/keyrings
wget -O /etc/apt/keyrings/grafana.asc \
  https://apt.grafana.com/gpg-full.key
chmod 644 /etc/apt/keyrings/grafana.asc
echo "deb [signed-by=/etc/apt/keyrings/grafana.asc] \
  https://apt.grafana.com stable main" \
  | tee /etc/apt/sources.list.d/grafana.list

apt-get update && apt-get install -y alloy
alloy --version
```

Configuration /etc/alloy/config.alloy

Alloy utilise le langage de configuration River (.alloy). La configuration définit des composants chaînés : source > traitement > destination.

```
// ---- Collecte Journal systemd ----
loki.source.journal "systemd" {
  forward_to = [loki.write.default.receiver]
  labels     = {
    host = "SRV-LOG",
    job  = "journal",
  }
}

// ---- Collecte fichiers syslog ----
local.file_match "syslog" {
  path_targets = [
    { __path__ = "/var/log/syslog",   job = "syslog" },
    { __path__ = "/var/log/auth.log", job = "auth" },
    { __path__ = "/var/log/kern.log", job = "kernel" },
  ]
}

loki.source.file "syslog" {
  targets    = local.file_match.syslog.targets
  forward_to = [loki.relabel.host.receiver]
}

// ---- Ajout du label host ----
loki.relabel "host" {
  forward_to = [loki.write.default.receiver]
  rule {
    action       = "replace"
    replacement  = "SRV-LOG"
    target_label = "host"
  }
}

// ---- Envoi vers Loki ----
loki.write "default" {
  endpoint {
    url = "http://localhost:3100/loki/api/v1/push"
  }
}
```

Permissions et démarrage

```
usermod -aG adm alloy
usermod -aG systemd-journal alloy

systemctl enable --now alloy
systemctl status alloy
```

Interface de debug

Alloy expose une interface web de debug sur le port 12345 permettant de visualiser le graphe des composants et vérifier que chaque pipeline est actif :

http://10.200.1.80:12345

Requêtes LogQL essentielles

Une fois les logs visibles dans Grafana > Explore > Loki, voici les requêtes de base :

Cas d'usage	Requête LogQL
Tous les logs journal	{job="journal"}
Logs d'un hôte précis	{host="SRV-CLOUD"}
Logs Docker Nextcloud	{container="nextcloud-app"}
Filtrer sur « error »	{job="journal"} |= "error"
Logs SSH (connexions)	{job="auth"} |= "Accepted"
Exclure un pattern	{job="syslog"} != "DEBUG"
Compter les erreurs/min	sum(rate({job="syslog"} |= "error" [1m]))
Logs Windows d'un serveur	{host="DC-IUTO5"}
Événements sécurité Windows	{host="DC-IUTO5", job="windows_events"}

Erreurs rencontrées et solutions

Erreur 1 : Erreur de format pour exec()

Symptôme : Loki refuse de démarrer avec une erreur de format d'exécution.

Cause : le binaire téléchargé ne correspond pas à l'architecture de la machine (ex: amd64 sur une machine ARM).

Solution :

```
uname -m   # Vérifier l'architecture
# Retélécharger le bon binaire selon le résultat
```

Erreur 2 : unknown short flag '-c'

Symptôme : Loki crash avec « unknown short flag '-c' ».

Cause : le service systemd utilise -config.file (simple tiret) au lieu de --config.file (double tiret).

Solution :

```
ExecStart=/usr/local/bin/loki -config.file=/etc/loki/config.yaml
ExecStart=/usr/local/bin/loki --config.file=/etc/loki/config.yaml
```

Erreur 3 : status=203/EXEC

Symptôme : systemd ne trouve pas le binaire.

Cause : le chemin dans ExecStart ne correspond pas à l'emplacement réel du binaire. Un paquet APT a installé Loki dans /opt/loki/ tandis que le binaire manuel est dans /usr/local/bin/.

Solution : vérifier avec which loki et corriger ExecStart dans le fichier service, ou supprimer le paquet en conflit avec apt remove loki.

Erreur 4 : logcli au lieu de loki

Symptôme : le binaire affiche « usage: logcli » au lieu de démarrer le serveur.

Cause : le mauvais fichier ZIP a été téléchargé (logcli au lieu de loki).

Solution : retélécharger loki-linux-amd64.zip (et non logcli-linux-amd64.zip).

Erreur 5 : unknown unit "h" in duration

Symptôme : Loki refuse de démarrer avec une erreur de parsing de durée.

Cause : un commentaire français avec des caractères accentués sur la même ligne qu'une valeur YAML perturbe le parser.

```
retention_period: 720h      # 30 jours de rétention
retention_period: 720h
```

Erreur 6 : delete-request-store should be configured

Symptôme : Loki crash car delete_request_store n'est pas défini.

Cause : Loki v3 exige cette option quand retention_enabled est activé.

Solution : ajouter delete_request_store: filesystem dans la section compactor.

Erreur 7 : allow_structured_metadata supprimé

Symptôme : Loki crash au démarrage.

Cause : l'option allow_structured_metadata a été supprimée en Loki v3.

Solution : supprimer cette ligne de la configuration.

Erreur 8 : relabel_rules = [] capsule error (Alloy)

Symptôme : Alloy refuse de démarrer avec « [] should be capsule, got array ».

Cause : le champ relabel_rules dans loki.source.journal n'accepte pas un tableau vide en syntaxe River.

Solution : supprimer la ligne relabel_rules = [] (le champ est optionnel).

Récapitulatif des fichiers importants

Fichier	Rôle
/usr/local/bin/loki	Binaire du serveur Loki
/etc/loki/config.yaml	Configuration de Loki
/etc/systemd/system/loki.service	Service systemd pour Loki
/var/lib/loki/	Données de stockage Loki (chunks, index, WAL)
/etc/alloy/config.alloy	Configuration d'Alloy (langage River)
/etc/default/alloy	Flags de démarrage du service Alloy
/var/lib/alloy/	Positions des fichiers lus par Alloy
C:\Program Files\GrafanaLabs\Alloy\config.alloy	Configuration Alloy sur Windows

Commandes de vérification

```
# Vérifier que Loki est prêt
curl http://localhost:3100/ready

# Lister les labels disponibles
curl -s http://localhost:3100/loki/api/v1/labels

# Lister les hôtes qui envoient des logs
curl -s http://localhost:3100/loki/api/v1/label/host/values

# Vérifier le statut de Loki
systemctl status loki

# Vérifier le statut de Grafana
systemctl status grafana-server

# Vérifier le statut d'Alloy
systemctl status alloy

# Voir les logs d'Alloy en temps réel
journalctl -u alloy -f

# Interface de debug Alloy
# http://10.200.1.80:12345

# Interface Grafana
# http://10.200.1.80:3000
```